

Day 38: Remove Last Transaction Using pop()

Day 38 — Remove Last Transaction Using pop()

1. Day Number

Day 38

2. Topic Name

List pop() Method

Today you will learn how to remove the **last item from a Python list** using:

```
pop()
```

3. Connection

Previously, you worked with lists such as a shopping cart.

Now imagine you are storing a **transaction history**:

```
Transaction 1  
Transaction 2  
Transaction 3
```

Sometimes the most recent transaction may need to be removed.

Today you will learn:

```
Transaction History  
    ↓  
Check if list has items  
    ↓  
Remove last transaction  
    ↓  
Updated history
```

The important idea is:

Check that the list is not empty before calling pop().

4. Important Topics

Today's main concepts are:

- Python list
- pop()
- checking whether a list is empty
- if condition
- returning or displaying the removed item

- function-based problem solving
-

5. Foundational Notes

What is pop()?

pop() is a built-in list method.

It removes an item from a list.

Without giving an index:

```
my_list.pop()
```

Python removes the **last item**.

Example list:

```
numbers = [10, 20, 30]
```

If pop() is used:

```
Before:  
[10, 20, 30]  
  
    pop()  
  
After:  
[10, 20]
```

The value 30 was removed.

pop() also returns the removed value

This is an important difference from some other list operations.

You can store the removed item:

```
removed_item = numbers.pop()
```

Conceptually:

```
[10, 20, 30]  
      ↓ pop()  
removed item = 30  
      ↓  
[10, 20]
```

So pop() does **two useful things**:

1. Removes an item
 2. Gives that removed item back
-

Why check for an empty list?

Suppose:

```
transactions = []
```

There is nothing available to remove.

Calling:

```
transactions.pop()
```

would produce an error.

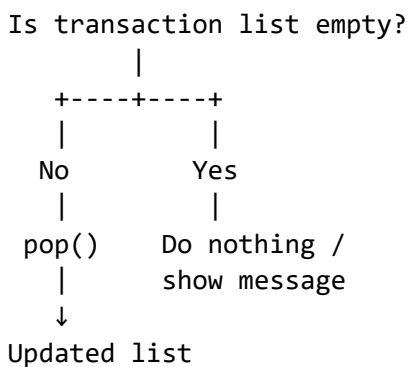
So first check:

```
if transactions:
```

This means:

If the list contains at least one item.

Conceptually:



pop() vs remove()

Yesterday you learned `remove()`.

They solve different problems.

remove()

Use it when you know the **value** you want to remove.

```
cart.remove("Laptop")
```

Meaning:

Find "Laptop" and remove it.

pop()

Use it when you want to remove an item based on its **position**.

```
transactions.pop()
```

Meaning:

| Remove the last item.

Simple comparison:

Method	Main idea
<code>remove(value)</code>	Remove a specific value
<code>pop()</code>	Remove the last item
<code>pop(index)</code>	Remove an item at a particular position

For today's exercise, concentrate only on:

```
pop()
```

6. Easy Example

Consider a list of tasks:

```
tasks = ["Study", "Exercise", "Shopping"]
```

You could remove the most recently added task using:

```
last_task = tasks.pop()
```

Afterward:

```
Removed:  
Shopping  
  
Remaining:  
["Study", "Exercise"]
```

But a safer pattern is:

```
if tasks:  
    # remove the last task
```

This prevents an error if `tasks` is empty.

This is only an example of the concept, **not today's complete solution**.

7. Problem Statement

Remove Last Transaction

Create a list containing some transaction names.

For example, the transactions could represent:

```
Deposit  
Shopping  
Electricity Bill
```

Your program should:

1. Create a transaction list.
2. Check whether the list contains any transactions.
3. If it contains transactions:
 - remove the **last transaction** using `pop()`
 - display the removed transaction
 - display the remaining transactions
4. If the list is empty:
 - print a suitable message such as:

```
No transactions available.
```

Important rule

Use `pop()` only when the list is **not empty**.

Do not provide a hardcoded solution specifically removing a named transaction.

The program should simply remove whichever transaction is currently last.

8. Concepts Used

You will practice:

List

Used to store multiple transactions.

```
transactions = [...]
```

`pop()`

Used to remove the last transaction.

```
transactions.pop()
```

`if`

Used to check whether a transaction exists before removing anything.

```
if transactions:
```

Variable

A variable can store the removed transaction.

Conceptually:

```
removed_transaction = last transaction
```

Function

You can place the transaction-removal logic inside a function.

Conceptually:

```
remove_last_transaction(...)
```

9. Thought Process

Before writing code, think about the problem step by step.

Step 1 — What data do I have?

A list of transactions.

Example:

```
["Deposit", "Shopping", "Electricity Bill"]
```

Step 2 — Which transaction should be removed?

Not a transaction selected by name.

You want the:

```
LAST transaction
```

Therefore `pop()` is suitable.

Step 3 — Can I always call `pop()`?

No.

The list could be:

```
[]
```

So first ask:

```
Does the list contain at least one transaction?
```

Step 4 — If yes

Remove the last transaction.

```
["Deposit", "Shopping", "Electricity Bill"]
                        ↑
                    remove
```

After removal:

```
["Deposit", "Shopping"]
```

Step 5 — What if the list is empty?

Do not call `pop()`.

Instead display something such as:

```
No transactions available.
```

So the overall thinking is:

```
Transactions
  ↓
Is list non-empty?
  ↓
+---+---+
|       |
| Yes   | No
|       |
| pop() | Message
|       |
|       |
| Store removed transaction
|       |
| Show updated list
```

10. Pseudocode

```
START

CREATE a function for removing the last transaction

    IF transaction list is not empty

        REMOVE the last transaction using pop()
        STORE the removed transaction

        DISPLAY removed transaction
        DISPLAY remaining transactions

    ELSE

        DISPLAY "No transactions available"

CREATE transaction list

CALL the function

END
```

Notice that the pseudocode describes the logic without giving you the complete Python implementation.

11. Suggested Solving Approach — Functional Approach

Use a small function to handle the removal logic.

Think of the structure as:

```
Main Program
  |
  v
Transaction List
  |
  v
remove_last_transaction()
  |
  v
Check empty?
 /    \
No      Yes
 |      |
pop()   Message
 |
Updated transactions
```

Your function could conceptually receive:

```
transactions
```

Then it should decide whether `pop()` can safely be used.

A useful mental model is:

```
Input
 ↓
Function
 ↓
Validation
 ↓
Operation
 ↓
Result
```

For this problem:

```
transaction list
 ↓
remove function
 ↓
empty-list check
 ↓
pop()
```


↓
updated transaction list

12. Easy Edge Cases

Edge Case 1 — Empty List

Input data:

```
transactions = []
```

There is nothing to remove.

Expected behavior:

```
No transactions available.
```

Your program should **not call** `pop()`.

Edge Case 2 — One Item List

Suppose:

```
["Deposit"]
```

After removing the last transaction:

```
Removed transaction: Deposit
```

The remaining list becomes:

```
[]
```

This is completely valid.

Edge Case 3 — Multiple Transactions

Suppose:

```
["Deposit", "Shopping", "Electricity Bill"]
```

Only:

```
Electricity Bill
```

should be removed.

Remaining transactions:

```
["Deposit", "Shopping"]
```

13. Expected Input

For this beginner exercise, you do not necessarily need `input()` from the keyboard.

You can start with a predefined list such as:

```
["Deposit", "Shopping", "Electricity Bill"]
```

Think of that list as your input data.

Another test input can be:

```
[]
```

And another:

```
["Deposit"]
```

Testing these three situations will help you understand the behavior.

14. Expected Output

For:

```
["Deposit", "Shopping", "Electricity Bill"]
```

Expected result could look like:

```
Removed transaction: Electricity Bill  
Remaining transactions: ['Deposit', 'Shopping']
```

For:

```
["Deposit"]
```

Expected result:

```
Removed transaction: Deposit  
Remaining transactions: []
```

For an empty list:

```
[]
```

Expected result:

```
No transactions available.
```

The exact wording can differ. The important thing is that the **logic is correct**.

15. Hint Only

Start by thinking about this structure:

```
def remove_last_transaction(transactions):  
    if ...:  
        removed_transaction = ...  
        # print removed transaction  
        # print remaining transactions  
    else:  
        # print empty-list message
```

Ask yourself three questions:

1. How can Python check whether a list contains at least one item?
2. Which list method removes and returns the last item?
3. Where should you call `pop()` so that an empty list never causes an error?

The central idea for **Day 38** is:

```
List  
↓  
Check not empty  
↓  
pop()  
↓  
Last item removed
```

Do not use `remove()` for today's exercise — practice `pop()` specifically.